Executive Summary: AI-Assisted CLO Database Analysis for Custodian Bank

Introduction

This report will summarise the completed design of an AI-assisted relational database for a Corporate Trust team within a custodian bank, supporting Collateralised Loan Obligations (CLOs). The proposal has identified the need to consolidate data relating to CLO deals, obligors, assets, trades, and portfolio tests to improve data quality, retrieval and analytical reporting.

The proposed solution combines ChatGPT, Python, SQLite, SQL and Power BI. ChatGPT was used to draft Python and SQL code and for the final executive level summary. Python was used to clean and transform the data with SQLite's relational tables, and lastly Power BI was used to create a dashboard and calculate the total KPIs.

Synthetic Dataset -> Python -> SQLite -> SQL -> Power BI Dashboard -> ChatGPT summary

Due to real world data confidentiality and data specifics, there are no publicly available suitable datasets on websites like Kaggle.com, so a synthetic CLO dataset was generated with the assistance of ChatGPT. The dataset was designed to reflect the structure of the proposed database and included deliberate data quality issues to support testing of the data cleaning step of the process.

The synthetic dataset was designed to replicate the type of data stored by custodian banks. It includes 41 columns and 600 rows of data, with each row representing one

CLO trade. It includes information about the trade itself (TradeID, Par Amount, Price, Amount, Currency, etc.), CLO deal information (DealID, DealName, Closing Date), collateral manager (ManagerID, ManagerName, ClientServiceLeadName), obligor (ObligorID, Obligor Name, Industries, Country), asset (AssetID, Asset Name, Ratings, Maturity etc.), and 3 portfolio tests – Class A OC Ratio, Class B OC Ratio, Interest Coverage Test (Test Threshold, Result, Headroom, Status).

Data Type	**Count**
Trades	600
CLO Deals	8
Collateral Managers	5
Obligors	40
Assets	90
Portfolio Tests	3

Data Preparation and Cleaning

ChatGPT was used to assist with writing Python code to clean data and identify missing values (see code in Appendix 1). Fields like TradeParAmount, TradePrice, TradeAmount were expected to be numeric, TradeDate, SettleDate, ClosingDate were expected to be valid dates, while DealName, ObligorName, AssetName were expected to be text. Identifiers such as TradeID, DealID, and AssetID were treated as text as well. Python was used to detect missing or invalid values, with numeric fields converted using pd.to_numeric(), which changed invalid values into null and dates converted using pd.to_datetime(). TradeType contained different variations such as “buy”, “Buy”, or “SELL”. These were standardised to the agreed values “Buy” and

“Sell”. Similarly, for industries, there were different variations like “Business Services” and “Business Services General”.

Incomplete records were investigated, and missing numerical values like TradeParAmount led to transactions being excluded, rather than replacing missing values with estimates. Transactions with unmatched DealID or AssetID, and duplicated TradeIDs were flagged as needing further investigation. This approach was chosen to reduce the risk of using incorrect information or, on the other hand, deleting legitimate information without proper evidence. This supports data cleaning principles as described by Sarkar and Roychowdhury (2019), where data preparation involves both correcting formats and controlled decision making about inconsistent and unsuitable data entries.

```
Rows: 600
Columns: 41

Missing values:
TradeParAmount    1
TradePrice        2
dtype: int64

Duplicate TradeIDs:
   TradeID DealID AssetID
30  T00031   D002    A035
31  T00031   D001    A089

Trade types:
TradeType
Buy     453
Sell    147
Name: count, dtype: int64

Deal IDs:
DealID
D003    72
D008    68
D005    61
D002    59
D999     1
Name: count, dtype: int64

Asset IDs:
AssetID
A008    2
A068    2
A047    2
A022    1
A999    1
Name: count, dtype: int64

Clean records: 593
Records with issues: 7

Total trade volume:
859000000.0

Buy vs Sell:
TradeType
Buy     660750000.0
Sell    198250000.0
Name: TradeParAmount, dtype: float64
```

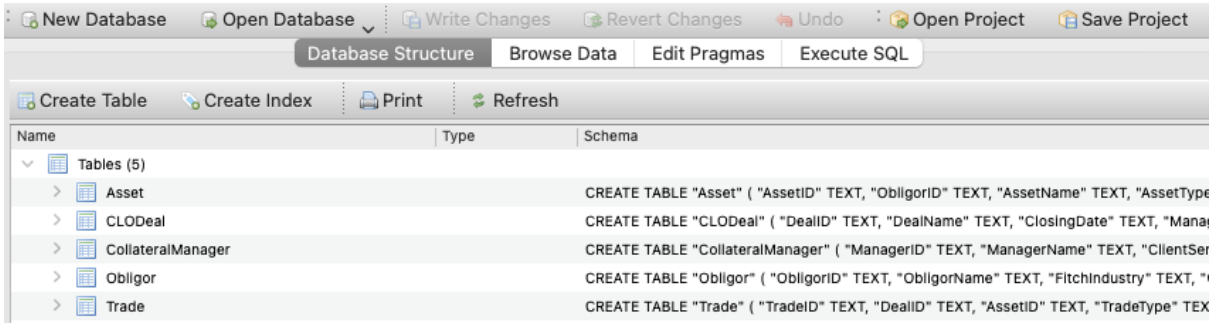
After cleaning, 593 trades remained for analysis (see Python output in Appendix 2). Seven records were excluded because they contained duplicated TradeIDs, missing

financial values or inconsistent DealID or AssetID values. The cleaned dataset covered eight CLO deals, 90 different assets and 40 obligors.

The total trade par value within the dataset was €859 million. Buy transactions represented €660.75 million, compared with €198.25 million of sell transactions, giving net traded par of €462.5 million. Trading activity was further analysed monthly and by industry and country. December had the highest trade volume at €124.25 million, and Technology recorded the highest trade par volume at €98.25 million. By country, Ireland had the highest activity at €126.25 million.

SQL Analysis

Following data cleaning in Python, the dataset was loaded into SQLite relational database. It was separated into five tables:



SQL queries (see Appendix 3) were used to validate and analyse the dataset by checking the total number of trades, comparing buy and sell total volumes, and analysing trade activity by industry and month.

SQL outputs were compared with Python results to confirm results remained unchanged, which provided an additional validation step before presenting the results in a Power BI dashboard.

Power BI Dashboard

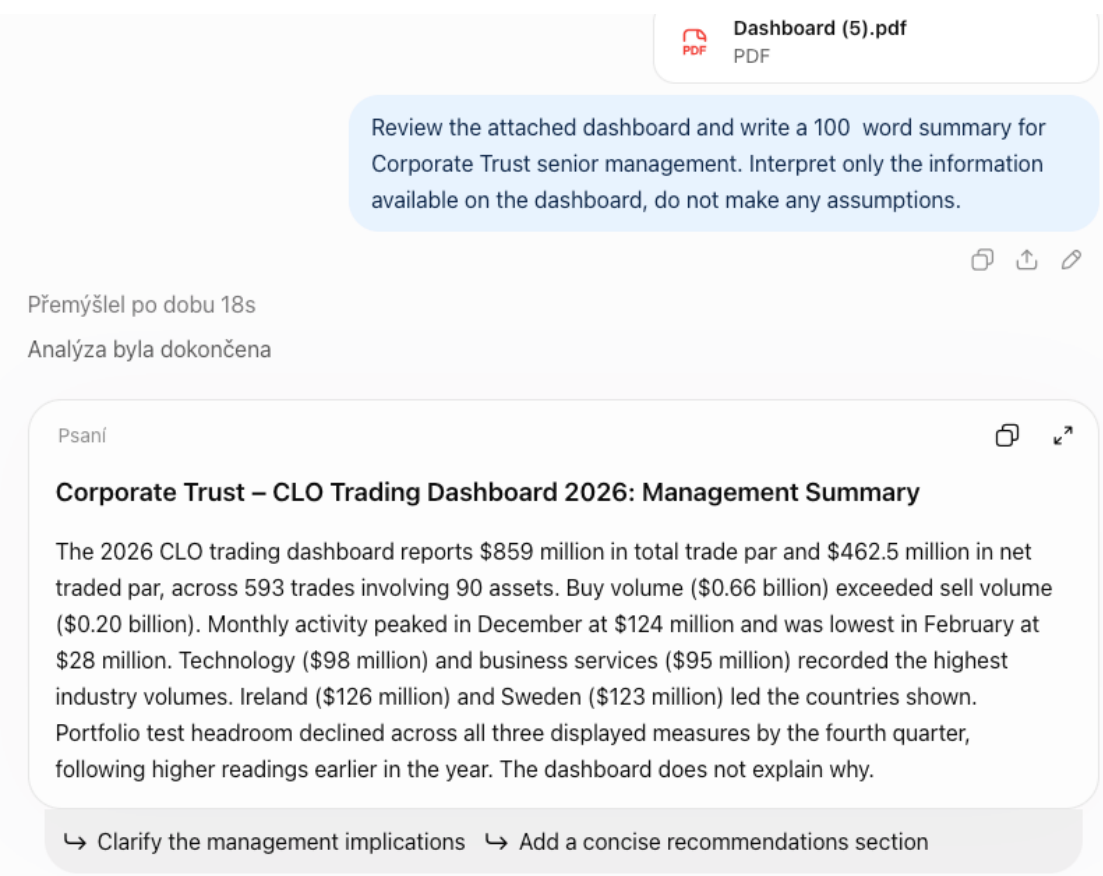
The cleaned dataset was then loaded into Power BI. Numerical data related to trades was converted into currency format, prices and spreads were turned into decimal numbers, and dates and text type data were checked. Power BI was selected as it allows combinations of summary indicators, charts and interactive filters on one dashboard.

The main measures were total trade par volume, net traded par, number of trades, and distinct assets traded. Supporting charts show trade volume by month, buy vs sell trade activity, trade volume by industry and country, and portfolio test headroom over time. The portfolio test headroom over time chart shows how much headroom there is between the test result and the threshold. Three slicers were added to make it easier to filter data by CLO deal, reporting period and currency (see filtered dashboard in Appendix 4).



AI Created Executive Summary

Using ChatGPT and the prompt “Review the attached dashboard and write a 100 word summary for Corporate Trust senior management. Interpret only the information available on the dashboard, do not make any assumptions.” the following executive summary for senior management was created based on the Power BI dashboard.



The AI output did not make any assumptions, and all numerical data provided in the summary is correct. Human interaction or providing the model with additional business or market context could lead to further explanation as to why, rather than simply presenting the main KPIs and other figures.

Evaluation of AI Supported Approach

The main benefit of using ChatGPT during the project was the speed at which it developed the initial Python code. It was particularly useful in suggesting ways

to detect missing or unsuitable values and standardising categories. It was also particularly useful for creating a senior management level summary out of the Power BI dashboard as a quick overview of Corporate Trust CLO operations.

However, this project also demonstrated why AI generated code requires human review. The code will run without an error even if it makes no sense from the business point of view. For example, AI may decide to automatically replace missing trade par value with an estimate or average, but this would be an unsuitable approach for real life financial data. That is why missing numerical data was flagged rather than used for analysis.

The same issue arises with CLO portfolio tests. CLO test calculations differ for each portfolio as it is set out in the specific portfolio legal documents and thresholds and logic also change based on which stage is the portfolio at. For example, during the Interest Payment Period, the Interest Coverage (IC) Tests are failing due to the interest being paid out to the investors. Without this additional knowledge, generative AI might incorrectly flag these fails as needing to be investigated rather than explaining why the fails are not completely valid indicators of performance.

These problems could potentially be resolved by training a specific AI agent to do the task with enhanced knowledge. Explicitly stating not to replace missing financial values with estimates or averages and training AI on deal documents outlining portfolio settings, its stages, and the effects on portfolio tests might decrease the chance of invalid outputs.

Even with these measures in place, human review will still be required at multiple stages throughout the dataset analysis. As explained in NIST's (2024) Generative AI Profile, there is a need to identify and manage risks associated with generative AI throughout its lifecycle. In this project, human review was applied through validation of generated code and separation of AI interpretation from underlying calculations.

Legal, GDPR, and Data Governance

Although synthetic data was used for this project, a real life Corporate Trust dataset would contain sensitive personal data and highly confidential commercial information. EU's General Data Protection Regulation (GDPR) sets out rules for handling data that relates to identifiable individuals. It requires organisations to protect personal information with appropriate technical and organisational measures and apply data minimisation and accuracy principles. Security measures appropriate to the level of risk need to be put in place as per Article 32 (EU, 2016).

Therefore, confidential client information cannot be entered into an unapproved public AI system. These controls are extremely important to maintain client confidentiality and prevent misuse of data. A partnership agreement, as seen between custodian bank BNY and generative AI producer OpenAI (BNY, 2026), can act as an additional layer of protection for confidential data.

Recommendations

The first business priority should be data quality and validation. Incoming files should be checked for missing values, duplicated identifiers, and invalid data types before being loaded into the dataset. Transactions failing these checks should be flagged and investigated, rather than letting AI automatically correct them.

The second priority should be human oversight. Engineers should review code produced by AI, and data analysts should check the database regularly, especially in the early stages of project implementation to ensure data accuracy. The portfolio tests in particular will need human review to check if failing tests are valid or if fails may be caused by the stage a portfolio is at.

The final priority should be controlled use of AI and treating data with confidentiality. ChatGPT can support code development and explain validated results, but business logic and financial calculations should remain subject to human review. Only organisationally approved AI environments should be used for analysing confidential data.

Conclusion

The completed prototype demonstrates how Python, SQLite, SQL, Power BI and ChatGPT can be combined to improve the analysis of CLO custodian bank data. Python AI generated code successfully identified missing and incorrect values and produced a cleaned dataset. ChatGPT helped with code development and interpretation, while Python, SQL and Power BI provided testable and comparable calculations and results. Overall, the proposed solution provides a structured and efficient approach to analysing Corporate Trust data while maintaining an appropriate level of human oversight, data quality and security. Further project development could focus on automated validation, exceptions reporting, and continued governed use of AI.

References:

BNY (2026) *Artificial Intelligence at BNY*. Available at: https://www.bny.com/corporate/global/en/about-us/technology-innovation/artificial-intelligence.html (Accessed: 2 October 2026).

BNY. (2026) *Structured Debt: CLO & Loan Solutions*. Available at: https://www.bny.com/corporate/global/en/solutions/platforms/structured-debt-solutions/clo-loan-solutions.html (Accessed 1 September 2026).

National Institute of Standards and Technology. (2024) *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. Available at: https://doi.org/10.6028/NIST.AI.600-1.

OpenAI. (2025) *BNY Builds "AI for Everyone, Everywhere" with OpenAI*. Available at: https://openai.com/index/bny/ (Accessed 1 September 2026).

Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation). (2016) *Official Journal L* 119, 4 May, pp. 1–88. Available at: https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng (Accessed: 6 October 2026).

Sarkar, T. and Roychowdhury, S. (2019) *Data Wrangling with Python*. Birmingham: Packt Publishing. Available at: https://www.oreilly.com/library/view/data-wrangling-with/9781789800111/ (Accessed 31 August 2026).

Appendices

Appendix 1 – ChatGPT Assisted Code

```
# ChatGPT Assisted Created Code
from pathlib import Path
import pandas as pd
import numpy as np

# Load dataset
DATA_FILE = Path("CLO_flat_dataset.csv")

df = pd.read_csv(DATA_FILE)

print("Rows:", len(df))
print("Columns:", len(df.columns))

# Check missing values
print("\nMissing values:")
print(df.isnull().sum()[df.isnull().sum() > 0])

# Check duplicate TradeIDs
print("\nDuplicate TradeIDs:")
print(df[df["TradeID"].duplicated(keep=False)][["TradeID", "DealID", "AssetID"]])

# Clean TradeType
df["TradeType"] = df["TradeType"].str.strip().str.title()

print("\nTrade types:")
print(df["TradeType"].value_counts())

# Convert numeric columns
numeric_columns = [
    "TradeParAmount",
    "TradePrice",
    "TradeAmount",
    "Spread",
    "Coupon"
```

```
]

for column in numeric_columns:
    df[column] = pd.to_numeric(df[column], errors="coerce")

# Convert dates
date_columns = [
    "ClosingDate",
    "MaturityDate",
    "TradeDate",
    "SettleDate"
]

for column in date_columns:
    df[column] = pd.to_datetime(df[column], errors="coerce")

# Standardise industry names
industry_map = {
    "Tech": "Technology",
    "TECH": "Technology",
    "TECHNOLOGY": "Technology",
    "Business Services General": "Business Services",
    "BUSINESS SERVICES": "Business Services",
    "Telecoms": "Telecommunications",
    "Industrial": "Industrials",
    "Transport": "Transportation",
    "Consumer": "Consumer Products"
}

df["Industry"] = df["FitchIndustry"].replace(industry_map)

# Check unusual DealIDs and AssetIDs
print("\nDeal IDs:")
print(df["DealID"].value_counts().tail())

print("\nAsset IDs:")
print(df["AssetID"].value_counts().tail())
```

```
# Flag bad records
df["DataIssue"] = False

df.loc[df["TradeParAmount"].isnull(), "DataIssue"] = True
df.loc[df["TradePrice"].isnull(), "DataIssue"] = True
df.loc[df["TradeID"].duplicated(keep=False), "DataIssue"] = True
df.loc[df["DealID"] == "D999", "DataIssue"] = True
df.loc[df["AssetID"] == "A999", "DataIssue"] = True

# Split clean and problem records
clean_df = df[df["DataIssue"] == False].copy()
problem_df = df[df["DataIssue"] == True].copy()

print("\nClean records:", len(clean_df))
print("Records with issues:", len(problem_df))

# Create new features
clean_df["SignedTradePar"] = np.where(
    clean_df["TradeType"] == "Buy",
    clean_df["TradeParAmount"],
    -clean_df["TradeParAmount"]
)

clean_df["SettlementDays"] = (
    clean_df["SettleDate"] - clean_df["TradeDate"]
).dt.days

clean_df["TradeMonth"] = clean_df["TradeDate"].dt.to_period("M").astype(str)

# Recalculate test headroom
clean_df["ClassA_Headroom_Check"] = (
    clean_df["ClassA_OC_Result"] -
    clean_df["ClassA_OC_Threshold"]
)

clean_df["ClassB_Headroom_Check"] = (
    clean_df["ClassB_OC_Result"] -
    clean_df["ClassB_OC_Threshold"]
```

```
)

clean_df["InterestCoverage_Headroom_Check"] = (
    clean_df["InterestCoverage_Result"] -
    clean_df["InterestCoverage_Threshold"]
)

# Simple summary
print("\nTotal trade volume:")
print(clean_df["TradeParAmount"].sum())

print("\nBuy vs Sell:")
print(
    clean_df.groupby("TradeType")["TradeParAmount"]
    .sum()
)

print("\nTrade volume by month:")
print(
    clean_df.groupby("TradeMonth")["TradeParAmount"]
    .sum()
)

print("\nTrade volume by industry:")
print(
    clean_df.groupby("Industry")["TradeParAmount"]
    .sum()
    .sort_values(ascending=False)
)

# Save files
clean_df.to_csv("CLO_clean_dataset.csv", index=False)
problem_df.to_csv("CLO_data_issues.csv", index=False)

print("\nFinished")
print("Clean file saved as: CLO_clean_dataset.csv")
print("Problem records saved as: CLO_data_issues.csv")
```

Appendix 2 – ChatGPT Assisted Code Output

Rows: 600

Columns: 41

Missing values:

TradeParAmount 1

TradePrice 2

dtype: int64

Duplicate TradeIDs:

TradeID DealID AssetID

30 T00031 D002 A035

31 T00031 D001 A089

Trade types:

TradeType

Buy 453

Sell 147

Name: count, dtype: int64

Deal IDs:

DealID

D003 72

D008 68

D005 61

D002 59

D999 1

Name: count, dtype: int64

Asset IDs:

AssetID

A008 2

A068 2

A047   2

A022   1

A999   1

Name: count, dtype: int64

Clean records: 593

Records with issues: 7

Total trade volume:

859000000.0

Buy vs Sell:

TradeType

Buy   660750000.0

Sell   198250000.0

Name: TradeParAmount, dtype: float64

Trade volume by month:

TradeMonth

2026-01   35000000.0

2026-02   27750000.0

2026-03   89500000.0

2026-04   57750000.0

2026-05   55250000.0

2026-06   67000000.0

2026-07   64500000.0

2026-08   72000000.0

2026-09   112250000.0

2026-10   73000000.0

2026-11   80750000.0

2026-12   124250000.0

Name: TradeParAmount, dtype: float64

Trade volume by industry:

Industry

Technology 98250000.0

Business Services 95000000.0

Transportation 86750000.0

Telecommunications 85500000.0

Consumer Products 84250000.0

Industrials 83500000.0

Specialty Chemicals 69750000.0

Healthcare 60500000.0

Media & Entertainment 46000000.0

Containers & Packaging 44000000.0

Media 35750000.0

Packaging 29500000.0

Chemicals 28250000.0

Health Care 12000000.0

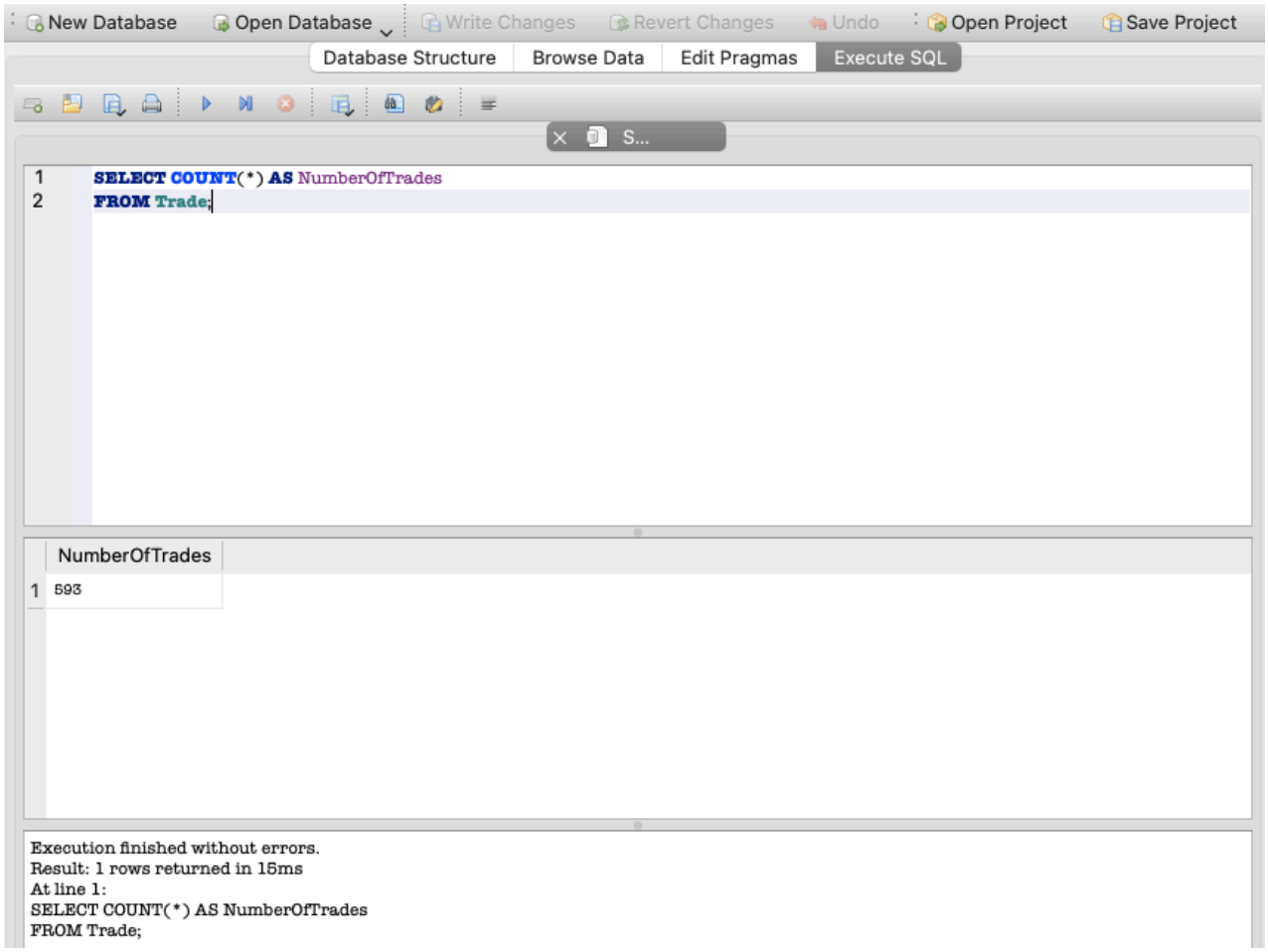
Name: TradeParAmount, dtype: float64

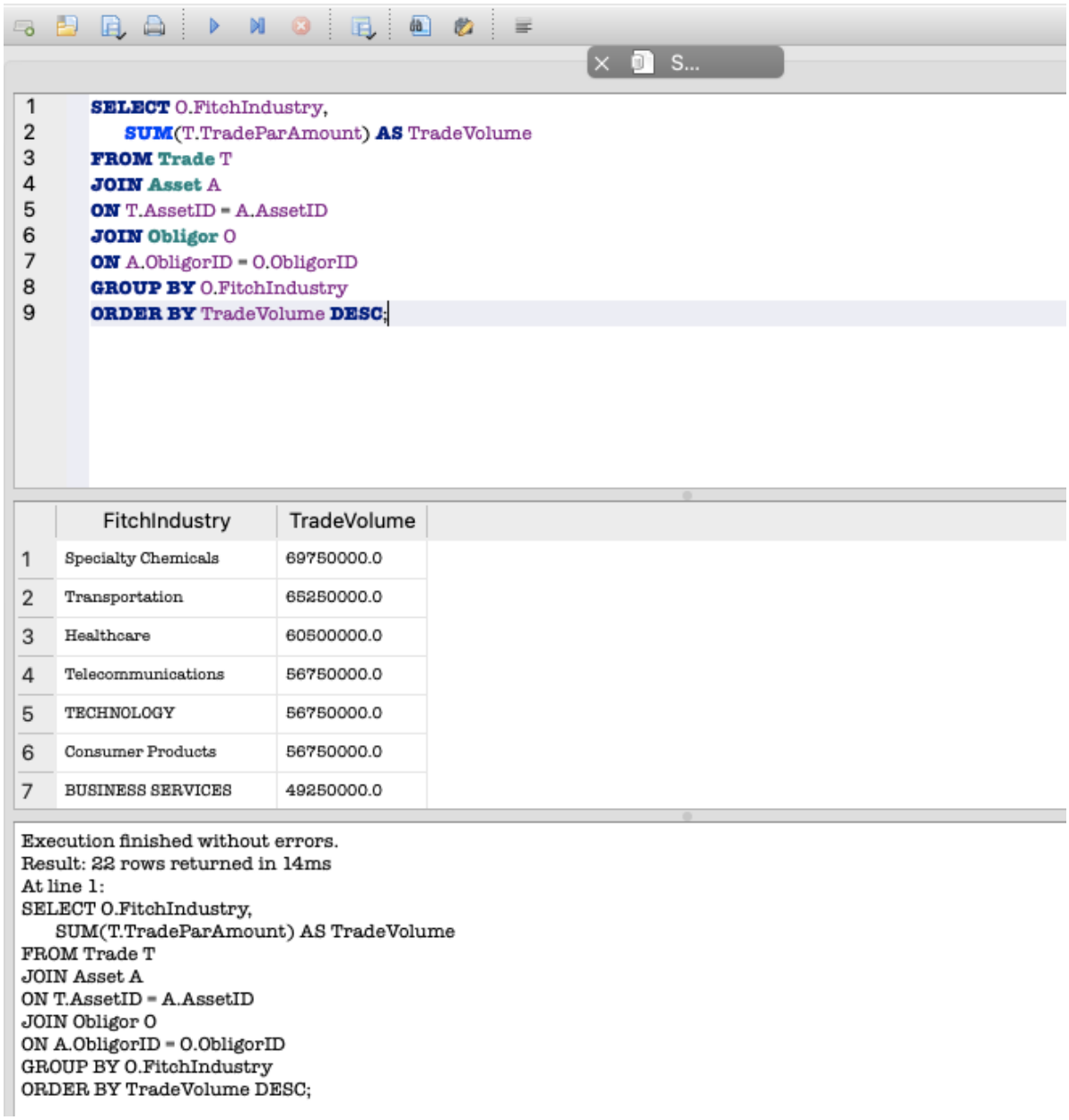
Finished

Clean file saved as: CLO_clean_dataset.csv

Problem records saved as: CLO_data_issues.csv

Appendix 3 – SQL Queries





```sql
SELECT O.FitchIndustry,
    SUM(T.TradeParAmount) AS TradeVolume
FROM Trade T
JOIN Asset A
ON T.AssetID = A.AssetID
JOIN Obligor O
ON A.ObligorID = O.ObligorID
GROUP BY O.FitchIndustry
ORDER BY TradeVolume DESC;
```

	FitchIndustry	TradeVolume
1	Specialty Chemicals	69750000.0
2	Transportation	65250000.0
3	Healthcare	60500000.0
4	Telecommunications	56750000.0
5	TECHNOLOGY	56750000.0
6	Consumer Products	56750000.0
7	BUSINESS SERVICES	49250000.0

```
Execution finished without errors.
Result: 22 rows returned in 14ms
At line 1:
SELECT O.FitchIndustry,
    SUM(T.TradeParAmount) AS TradeVolume
FROM Trade T
JOIN Asset A
ON T.AssetID = A.AssetID
JOIN Obligor O
ON A.ObligorID = O.ObligorID
GROUP BY O.FitchIndustry
ORDER BY TradeVolume DESC;
```

Appendix 4 – Dashboard filtered for Jun-Dec, USD Currency, Ireland

